AI

AI coding tools may not speed up every developer, study shows

Maxwell Zeff — 12:56 PM PDT · July 11, 2025

Software engineer workflows have been transformed in recent years by an influx of AI coding tools like Cursor and GitHub Copilot, which promise to enhance productivity by automatically writing lines of code, fixing bugs, and testing changes. The tools are powered by AI models from OpenAI, Google DeepMind, Anthropic, and xAI that have rapidly increased their performance on a range of software engineering tests in recent years.

However, a new study published Thursday by the non-profit AI research group METR calls into question the extent to which today's AI coding tools enhance productivity for experienced developers.

IMAGE CREDITS: YUICHIRO CHINO / GETTY IMAGES

METR conducted a randomized controlled trial for this study by recruiting 16 experienced open source developers and having them complete 246 real tasks on large code repositories they regularly contribute to. The researchers randomly assigned roughly half of those tasks as "AI-allowed," giving developers permission to use state-of-the-art AI coding tools such as Cursor Pro, while the other half of tasks forbade the use of AI tools.

Before completing their assigned tasks, the developers forecasted that using AI coding tools would reduce their completion time by 24%. That wasn't the case.

"Surprisingly, we find that allowing AI actually increases completion time by 19% — developers are slower when using AI tooling," the researchers said.

Notably, only 56% of the developers in the study had experience using Cursor, the main AI tool offered in the study. While nearly all the developers (94%) had experience using some web-based LLMs in their coding workflows, this study was the first time some used Cursor specifically. The researchers note that developers were trained on using Cursor in preparation for the study.

Nevertheless, METR's findings raise questions about the supposed universal productivity gains promised by AI coding tools in 2025. Based on the study, developers shouldn't assume that AI coding tools — specifically what's come to be known as "vibe coders" — will immediately speed up their workflows.

METR researchers point to a few potential reasons why AI slowed down developers rather than speeding them up: Developers spend much more time prompting AI and waiting for it to respond when using vibe coders rather than actually coding. AI also tends to struggle in large, complex code bases, which this test used.

The study's authors are careful not to draw any strong conclusions from these findings, explicitly noting they don't believe AI systems currently fail to speed up many or most software developers. Other large-scale studies have shown that AI coding tools do speed up software engineer workflows.

The authors also note that AI progress has been substantial in recent years and that they wouldn't expect the same results even three months from now. METR has also found that AI coding tools have significantly improved their ability to complete complex, long-horizon tasks in recent years.

However, the research offers yet another reason to be skeptical of the promised gains of AI coding tools. Other studies have shown that today's AI coding tools can introduce mistakes and, in some cases, security vulnerabilities.

Topics: AI ai coding cursor

Maxwell Zeff
Senior AI Reporter

Maxwell Zeff is a senior reporter at TechCrunch specializing in AI. Previously with Gizmodo, Bloomberg,...

View Bio

KEEP READING

AI

Google unveils Gemini CLI, an open source AI tool for terminals

developers are already coding.

The company announced on Wednesday the launch of Gemini CLI, an agentic AI tool designed to run locally from your terminal. The new tool connects Google's Gemini AI models to local codebases, and it allows developers to make natural language requests, such as asking Gemini CLI to explain confusing sections of code, write new features, debug code, or run commands.

Gemini CLI is part of Google's efforts to get developers using its AI models in their coding workflows. Google now offers an array of AI coding tools, such as Gemini Code Assist and its asynchronous AI coding assistant, Jules. However, Gemini CLI competes directly with other command-line AI tools such as OpenAI's Codex CLI and Anthropic's Claude Code — tools that tend to be easier to integrate, faster, and more efficient than other AI coding tools.

Since Google launched Gemini 2.5 Pro in April, the company's AI models have become a favorite among developers. The popularity of Gemini 2.5 Pro has driven usage of third-party AI coding tools, such as Cursor and GitHub Copilot, which have become massive businesses. In response, Google has tried in recent months to build a direct relationship with these developers by offering in-house products.

While most people will use Gemini CLI for coding, the company says it designed the tool to handle other tasks as well. Developers can tap Gemini CLI to create videos with Google's Veo 3 model, generate research reports with the company's Deep Research agent, or access real-time information through Google Search. Google also says Gemini CLI can connect to MCP servers, allowing developers to connect to external databases.

To encourage adoption, Google is also open sourcing Gemini CLI under the Apache 2.0 license, which is typically considered one of the most permissive. The company says it expects a network of developers to contribute to the project on GitHub.

Google is also offering generous usage limits to spur adoption of Gemini CLI. Free users can make 60 model requests per minute and 1,000 requests per day, which the company says is roughly double the average number of requests developers made when using the tool.

While AI coding tools are rising rapidly in popularity, using them comes with risks. According to a 2024 survey from Stack Overflow, just 43% of developers trust the accuracy of AI tools. Several studies have shown that code-generating AI models can occasionally introduce errors or fail to fix security vulnerabilities.

Topics: AI ai coding Google

Maxwell Zeff

Senior AI Reporter |

Maxwell Zeff is a senior reporter at TechCrunch specializing in AI. Previously with Gizmodo, Bloomberg,...

View Bio >

KEEP READING

IMAGE CREDITS: VGAJIC / GETTY IMAGES

AI

OpenAI's Codex is part of a new cohort of agentic coding tools

Russell Brandom — 5:30 AM PDT · May 20, 2025

Last Friday, OpenAI introduced a new coding system called Codex, designed to perform complex programming tasks from natural language commands. Codex moves OpenAI into a new cohort of agentic coding tools that is just beginning to take shape.

Latest in AI

See More

From GitHub's early Copilot to contemporary tools like Cursor and Windsurf, most AI coding assistants operate as an exceptionally intelligent form of autocomplete. The tools generally live in an integrated development environment, and users interact directly with the AI-generated code. The prospect of simply assigning a task and returning when it's finished is largely out of reach.

But these new agentic coding tools, led by products like Devin, SWE-Agent, OpenHands, and OpenAI Codex, are designed to work without users ever having to see the code. The goal is to operate like the manager of an engineering team, assigning issues through workplace systems like Asana or Slack and checking in when a solution has been reached.

For believers in forms of highly capable AI, it's the next logical step in a natural progression of automation taking over more and more software work.

"In the beginning, people just wrote code by pressing every single keystroke," explains Kilian Lieret, a Princeton researcher and member of the SWE-Agent team. "GitHub Copilot was the first product that offered real auto-complete, which is kind of stage two. You're still absolutely in the loop, but sometimes you can take a shortcut."

The goal for agentic systems is to move beyond developer environments entirely, instead presenting coding agents with an issue and leaving them to resolve it on their own. "We pull things back to the management layer, where I just assign a bug report and the bot tries to fix it completely autonomously," says Lieret.

It's an ambitious aim, and so far, it's proven difficult.

After Devin became generally available at the end of 2024, it drew scathing criticism from YouTube pundits, as well as a more measured critique from an early client at Answer.AI. The overall impression was a familiar one for vibe-coding veterans: with so many errors, overseeing the models takes as much work as doing the task manually. (While Devin's rollout has been a bit rocky, it hasn't stopped fundraisers from recognizing the potential — in March, Devin's parent company, Cognition AI, reportedly raised hundreds of millions of dollars at a $4 billion valuation.)

Even supporters of the technology caution against unsupervised vibe-coding, seeing the new coding agents as powerful elements in a human-supervised development process.

"Right now, and I would say, for the foreseeable future, a human has to step in at code review time to look at the code that's been written," says Robert Brennan, the CEO of All Hands AI, which maintains OpenHands. "I've seen several people work themselves into a mess by just auto-approving every bit of code that the agent writes. It gets out of hand fast."

Hallucinations are an ongoing problem as well. Brennan recalls one incident in which, when asked about an API that had been released after the OpenHands agent's training data cutoff, the agent fabricated details of an API that fit the description. All Hands AI says it's working on systems to catch these hallucinations before they can cause harm, but there isn't a simple fix.

Arguably the best measure of agentic programming progress is the SWE-Bench leaderboards, where developers can test their models against a set of unresolved issues from open GitHub repositories. OpenHands currently holds the top spot on the verified leaderboard, solving 65.8% of the problem set. OpenAI claims that one of the models powering Codex, codex-1, can do better, listing a 72.1% score in its announcement — although the score came with a few caveats and hasn't been independently verified.

The concern among many in the tech industry is that high benchmark scores don't necessarily translate to truly hands-off agentic coding. If agentic coders can only solve three out of every four problems, they're going to require significant oversight from human developers — particularly when tackling complex systems with multiple stages.

Like most AI tools, the hope is that improvements to foundation models will come at a steady pace, eventually enabling agentic coding systems to grow into reliable developer tools. But finding ways to manage hallucinations and other reliability issues will be crucial for getting there.

"I think there is a little bit of a sound barrier effect," Brennan says. "The question is, how much trust can you shift to the agents, so they take more out of your workload at the end of the day?"

Topics: AI vibe coding

Russell Brandom

Russell is a freelance writer based in New York.

View Bio